02-13、接口自动化测试Agent实战—工具封装+决策引擎开发

一、数据库查询工具

1、接口查询工具

```
from typing import List, Dict, Any, Optional, Type
from langchain.tools import BaseTool
from pydantic import BaseModel, Field
import json
from db.mysql_database import MysqlDB


class InterfaceQueryInput(BaseModel):
    """接口查询工具的输入参数"""
    project: Optional[str] = Field(None, description="项目名称，如'web测试平台'")
    module: Optional[str] = Field(None, description="模块名称，如'用户认证'、'支付管理'")
    interface_name: Optional[str] = Field(None, description="接口名称，支持模糊匹配")
    tag: Optional[str] = Field(None, description="接口分类标签")
    method: Optional[str] = Field(None, description="HTTP请求方法：GET/POST/PUT/DELETE/PATCH")

class InterfaceQueryTool(BaseTool):
    """接口查询工具"""
    name: str = "interface_query"
    description: str = """
        接口查询工具，用于查询API接口信息。

        功能：
        - 根据项目、模块、接口名称、标签、请求方法等条件查询接口
        - 支持接口名称的模糊匹配
        - 支持多条件组合查询
        - 自动处理JSON字段的序列化和反序列化
        - 返回格式化的查询结果

        使用场景：
        - 获取特定项目的所有接口
        - 获取特定模块的接口列表
        - 获取特定名称的接口
        - 获取特定标签的接口集合
        - 获取特定请求方法的接口

        输入参数：
        - project: 项目名称（可选）
        - module: 模块名称（可选）
        - interface_name: 接口名称（可选，支持模糊匹配）
        - tag: 接口标签（可选）
        - method: 请求方法（可选）
```

```
        """
    args_schema: Type[BaseModel] = InterfaceQueryInput

    def _run(self, project: Optional[str] = None, module: Optional[str] = None,
             interface_name: Optional[str] = None, tag: Optional[str] = None,
             method: Optional[str] = None) -> str:
        """
        执行接口查询
        :param project: 项目名称
        :param module: 模块名称
        :param interface_name: 接口名称
        :param tag: 接口标签
        :param method: 请求方法
        :return: 查询结果（JSON字符串）
        """
        try:
            with MysqlDB() as db:
                query = "SELECT * FROM api_interface WHERE is_delete = 0"
                params = []

                if project:
                    query += " AND project = %s"
                    params.append(project)
                if module:
                    query += " AND module = %s"
                    params.append(module)
                if interface_name:
                    query += " AND (summary LIKE %s OR description LIKE %s)"
                    params.extend([f"%{interface_name}%", f"%
{interface_name}%"])
                if tag:
                    query += " AND tag LIKE %s"
                    params.append(f"%{tag}%")
                if method:
                    query += " AND method = %s"
                    params.append(method)

                db.cursor.execute(query, params)
                results = db.cursor.fetchall()

                # 将JSON字段转换为Python对象
                for result in results:
                    for field in ['parameters', 'requestBody', 'responses',
'security']:
                        if result.get(field):
                            result[field] = json.loads(result[field])

                return json.dumps(results, ensure_ascii=False, indent=2)

        except Exception as e:
            return f"查询出错: {str(e)}"
```

2、基础用例查询工具

```
from typing import List, Dict, Any, Optional, Type
```

```python
from langchain.tools import BaseTool
from pydantic import BaseModel, Field
import json
from db.mysql_database import MysqlDB

class BaseTestCaseQueryInput(BaseModel):
    """基础用例查询工具的输入参数"""
    project: Optional[str] = Field(None, description="项目名称")
    module: Optional[str] = Field(None, description="模块名称")
    interface_name: Optional[str] = Field(None, description="接口名称，支持模糊匹
配")
    interface_id: Optional[str] = Field(None, description="接口ID，关联
api_interface表")
    status: Optional[str] = Field(None, description="用例状态：
pending/generated/failed")


class BaseTestCaseQueryTool(BaseTool):
    """基础用例查询工具"""
    name: str = "base_test_case_query"
    description: str = """
        基础用例查询工具，用于查询基础测试用例信息。

        功能：
        - 根据项目、模块、接口名称、接口ID、状态等条件查询基础用例
        - 支持接口名称的模糊匹配
        - 支持多条件组合查询
        - 自动处理JSON字段的序列化和反序列化
        - 返回格式化的查询结果

        使用场景：
        - 获取特定项目的所有基础用例
        - 获取特定模块的用例列表
        - 获取特定接口的用例集合
        - 获取特定状态的用例集合
        - 用例生成和审核过程中的查询

        输入参数：
        - project: 项目名称（可选）
        - module: 模块名称（可选）
        - interface_name: 接口名称（可选，支持模糊匹配）
        - interface_id: 接口ID（可选）
        - status: 用例状态（可选）
        """
    args_schema: Type[BaseModel] = BaseTestCaseQueryInput

    def _run(self, project: Optional[str] = None, module: Optional[str] = None,
             interface_name: Optional[str] = None, interface_id: Optional[str] =
None,
             status: Optional[str] = None) -> str:
        """
        执行基础用例查询
        :param project: 项目名称
        :param module: 模块名称
        :param interface_name: 接口名称
        :param interface_id: 接口ID
        :param status: 用例状态
        :return: 查询结果（JSON字符串）
```

```
        """
        try:
            with MysqlDB() as db:
                query = """
                    SELECT bc.*, ai.summary as interface_summary, ai.module as
interface_module
                    FROM base_test_case bc
                    LEFT JOIN api_interface ai ON bc.interface_id =
ai.interface_id
                    WHERE 1=1
                """
                params = []

                if project:
                    query += " AND bc.project = %s"
                    params.append(project)
                if module:
                    query += " AND ai.module = %s"
                    params.append(module)
                if interface_name:
                    query += " AND ai.summary LIKE %s"
                    params.append(f"%{interface_name}%")
                if interface_id:
                    query += " AND bc.interface_id = %s"
                    params.append(interface_id)
                if status:
                    query += " AND bc.status = %s"
                    params.append(status)

                db.cursor.execute(query, params)
                results = db.cursor.fetchall()

                # 将JSON字段转换为Python对象
                for result in results:
                    for field in ['steps', 'expected', 'dependencies']:
                        if result.get(field):
                            result[field] = json.loads(result[field])

                return json.dumps(results, ensure_ascii=False, indent=2)

        except Exception as e:
            return f"查询出错: {str(e)}"
```

3、结构化查询工具

```
# @Author  : 木森
# @weixin: python771
"""
数据库查询工具集
包含接口查询、基础用例查询和结构化用例查询工具
"""

from typing import List, Dict, Any, Optional, Type
from langchain.tools import BaseTool
from pydantic import BaseModel, Field
import json
```

```python
from db.mysql_database import MysqlDB



class StructuredTestCaseQueryInput(BaseModel):
    """结构化用例查询工具的输入参数"""
    project: Optional[str] = Field(None, description="项目名称")
    module: Optional[str] = Field(None, description="模块名称")
    interface_name: Optional[str] = Field(None, description="接口名称，支持模糊匹配")
    base_case_id: Optional[str] = Field(None, description="基础用例ID，关联base_test_case表")
    review_status: Optional[str] = Field(None, description="审核状态：pending/通过/失败/错误")
    include_base_case: Optional[bool] = Field(False, description="是否包含关联的基础用例信息")
    include_interface: Optional[bool] = Field(False, description="是否包含关联的接口信息")


class StructuredTestCaseQueryTool(BaseTool):
    """结构化用例查询工具"""
    name: str = "structured_test_case_query"
    description: str = """
        结构化用例查询工具，用于查询结构化测试用例信息。

        功能：
        - 根据项目、模块、接口名称、基础用例ID、审核状态等条件查询结构化用例
        - 支持接口名称的模糊匹配
        - 支持按模块、接口ID进行关联查询
        - 支持包含关联的基础用例和接口信息
        - 支持多条件组合查询
        - 自动处理JSON字段的序列化和反序列化
        - 返回格式化的查询结果

        使用场景：
        - 获取特定项目的所有结构化用例
        - 获取特定模块的用例列表
        - 获取特定接口的用例集合
        - 获取特定基础用例的结构化用例
        - 获取特定审核状态的用例集合
        - 用例审核和执行过程中的查询
        - 需要查看用例完整上下文信息的场景

        输入参数：
        - project: 项目名称（可选）
        - module: 模块名称（可选）
        - interface_name: 接口名称（可选，支持模糊匹配）
        - base_case_id: 基础用例ID（可选）
        - review_status: 审核状态（可选）
        - include_base_case: 是否包含基础用例信息（可选，默认False）
        - include_interface: 是否包含接口信息（可选，默认False）
        """
    args_schema: Type[BaseModel] = StructuredTestCaseQueryInput

    def _run(self, project: Optional[str] = None, module: Optional[str] = None,
             interface_name: Optional[str] = None, base_case_id: Optional[str] = None,
```

```
             review_status: Optional[str] = None, include_base_case: bool = False,
             include_interface: bool = False) -> str:
        """
        执行结构化用例查询
        :param project: 项目名称
        :param module: 模块名称
        :param interface_name: 接口名称
        :param base_case_id: 基础用例ID
        :param review_status: 审核状态
        :param include_base_case: 是否包含基础用例信息
        :param include_interface: 是否包含接口信息
        :return: 查询结果（JSON字符串）
        """
        try:
            with MysqlDB() as db:
                # 构建基础查询
                query = """
                    SELECT stc.*,
                           ai.interface_id,
                           ai.summary as interface_summary,
                           ai.module as interface_module
                    FROM structured_test_case stc
                    LEFT JOIN base_test_case bc ON stc.base_case_id = bc.case_id
                    LEFT JOIN api_interface ai ON bc.interface_id = ai.interface_id
                    WHERE 1=1
                """
                params = []

                # 添加查询条件
                if project:
                    query += " AND stc.project = %s"
                    params.append(project)
                if module:
                    query += " AND (stc.module = %s OR ai.module = %s)"
                    params.extend([module, module])
                if interface_name:
                    query += " AND ai.summary LIKE %s"
                    params.append(f"%{interface_name}%")
                if base_case_id:
                    query += " AND stc.base_case_id = %s"
                    params.append(base_case_id)
                if review_status:
                    query += " AND stc.review_status = %s"
                    params.append(review_status)

                # 执行查询
                db.cursor.execute(query, params)
                results = db.cursor.fetchall()

                # 处理结果
                processed_results = []
                for result in results:
                    # 处理JSON字段
                    for field in ['preconditions', 'request', 'assertions']:
                        if result.get(field):
                            result[field] = json.loads(result[field])
```

```
                    # 处理基础用例信息
                    if include_base_case:
                        base_case_query = "SELECT * FROM base_test_case WHERE
case_id = %s"
                        db.cursor.execute(base_case_query,
[result['base_case_id']])
                        base_case = db.cursor.fetchone()
                        if base_case:
                            for field in ['steps', 'expected', 'dependencies']:
                                if base_case.get(field):
                                    base_case[field] =
json.loads(base_case[field])
                            result['base_case'] = base_case

                    # 处理接口信息
                    if include_interface:
                        interface_query = "SELECT * FROM api_interface WHERE
interface_id = %s"
                        db.cursor.execute(interface_query,
[result['interface_id']])
                        interface = db.cursor.fetchone()
                        if interface:
                            for field in ['parameters', 'requestBody',
'responses', 'security']:
                                if interface.get(field):
                                    interface[field] =
json.loads(interface[field])
                            result['interface'] = interface

                    processed_results.append(result)

                return json.dumps(processed_results, ensure_ascii=False,
indent=2)

        except Exception as e:
            return f"查询出错: {str(e)}"


if __name__ == '__main__':
    # 测试接口查询工具
    interface_tool = InterfaceQueryTool()
    interface_result = interface_tool.run(
        project="web测试平台",
        module="用户认证",
        interface_name="登录"
    )
    print("接口查询结果:", interface_result)

    # 测试基础用例查询工具
    base_case_tool = BaseTestCaseQueryTool()
    base_case_result = base_case_tool.run(
        project="web测试平台",
        module="用户认证",
        interface_name="登录"
    )
    print("基础用例查询结果:", base_case_result)
```

```
    # 测试结构化用例查询工具
    structured_case_tool = StructuredTestCaseQueryTool()
    # 测试基本查询
    basic_result = structured_case_tool.run(
        project="web测试平台",
        module="用户认证",
        interface_name="登录"
    )
    print("基本查询结果:", basic_result)

    # 测试包含关联信息的查询
    full_result = structured_case_tool.run(
        project="web测试平台",
        module="用户认证",
        interface_name="登录",
        include_base_case=True,
        include_interface=True
    )
    print("完整查询结果:", full_result)
```

二、用例执行工具封装

1、接口测试用例执行工具

```
# @Author  : 木森
# @weixin: python771
"""
用例执行工具
用于执行结构化测试用例并返回执行结果
"""

import os
import json
from typing import Dict, Any, Optional, Type, List
from langchain.tools import BaseTool
from pydantic import BaseModel, Field
from api_auto_test.main import run_test


class CaseExecutionInput(BaseModel):
    """用例执行工具的输入参数"""
    test_data: List[Dict[str, Any]] = Field(description="测试用例数据集")


class CaseExecutionTool(BaseTool):
    """用例执行工具"""
    name: str = "case_execution"
    description: str = """
        用例执行工具，用于执行结构化测试用例并返回执行结果。

        功能:
        - 执行测试用例数据集
        - 自动生成测试报告
```

```
        - 返回详细的执行结果和错误信息

        使用场景:
        - 执行单个测试用例
        - 执行多个测试用例
        - 执行特定项目的用例集合
        - 执行特定模块的用例集合
        - 执行特定接口的用例集合
        - 测试执行和结果分析

        输入参数:
        - test_data: 测试用例数据集（必需）
            [
                {
                    "case_id": "用例ID",
                    "name": "用例名称",
                    "description": "用例描述",
                    "project": "项目名称",
                    "module": "模块名称",
                    "interface_name": "接口名称",
                    "preconditions": ["前置条件"],
                    "request": {"请求数据"},
                    "assertions": ["断言"]
                }
            ]
        """
    args_schema: Type[BaseModel] = CaseExecutionInput

    def _run(self, test_data: List[Dict[str, Any]]) -> str:
        """
        执行测试用例
        :param test_data: 测试用例数据集
        :return: 执行结果（JSON字符串）
        """
        try:
            # 保存当前工作路径
            old_path = os.getcwd()
            # 切换到api_auto_test目录
            os.chdir(os.path.join(os.path.dirname(os.path.dirname(__file__)),
'api_auto_test'))
            result = run_test(test_data)
            os.chdir(old_path)
        except Exception as e:
            return json.dumps({
                "status": "error",
                "message": f"执行出错: {str(e)}"
            }, ensure_ascii=False, indent=2)
        else:
                return json.dumps(result, ensure_ascii=False, indent=2)
            finally:
                # 恢复工作路径

if __name__ == '__main__':
    # 测试用例执行工具
    tool = CaseExecutionTool()

    test_data = [
        {
```

```
        "test_case_id": "TC001",
        "name": "用户登录功能测试（用户名+密码）",
        "description": "验证使用正确用户名和密码登录的功能",
        "project": "web测试平台",
        "module": "用户模块",
        "interface": "用户登录接口",
        "tags": ["登录", "认证"],

        "preconditions": [
            {
                "name": "注册测试用户",
                "request": {
                    "interface_id": "register_001",
                    "method": "POST",
                    "url": "/api/users/register",
                    "base_url": "http://124.222.144.182:8888",
                    "headers": {
                        "Content-Type": "application/json"
                    },
                    "body": {
                        "username": "$generate_username(prefix=t)",
                        "password": "12345678qwe",
                        "password_confirm": "12345678qwe",
                        "email": "$generate_email(prefix=t)",
                        "mobile": "$generate_mobile()",
                        "nickname": "$generate_nickname()"
                    }
                },
                "extract": [
                    ["username", "username"],  # 提取生成的用户名
                    ["email", "email"],  # 提取生成的邮箱
                    ["mobile", "mobile"]  # 提取生成的手机号
                ]
            }
        ],

        "request": {
            "interface_id": "login_001",
            "name": "用户登录请求",
            "method": "POST",
            "url": "/api/users/login",
            "base_url": "http://124.222.144.182:8888",
            "headers": {
                "Content-Type": "application/json"
            },
            "body": {
                "username": "{{username}}",  # 使用前置步骤中提取的用户名
                "password": "12345678qwe"
            }
        },

        "assertions": {
            "response": [
                {
                    "type": "http_code",
                    "field": "HTTP状态码",
                    "expected": 200
                },
```

```
                {
                    "type": "equal",
                    "field": "user.username",
                    "expected": "{{username}}"  # 使用提取的用户名进行断言
                },
                {
                    "type": "equal",
                    "field": "user.email",
                    "expected": "{{email}}"  # 使用提取的邮箱进行断言
                },
                {
                    "type": "equal",
                    "field": "user.mobile",
                    "expected": "{{mobile}}"  # 使用提取的手机号进行断言
                }
            ],
            "database": []
        }
    },
    {
        "name": "创建项目功能测试（正常场景）",
        "description": "测试创建项目的正常流程，验证响应数据和状态码",
        "project": "web测试平台",
        "module": None,
        "interface": "创建项目",
        "preconditions": [
            {
                "name": "用户登录",
                "request": {
                    "interface_id": "用户登录",
                    "method": "POST",
                    "url": "/api/users/login",
                    "base_url": "http://124.222.144.182:8888",
                    "headers": {
                        "Content-Type": "application/json"
                    },
                    "params": {},
                    "body": {
                        "username": "python771",
                        "password": "musen12399"
                    }
                },
                "extract": [
                    ["token", "token"],
                    ["user_id", "user.id"]
                ]
            }
        ],
        "request": {
            "interface_id": "创建项目",
            "name": "创建项目请求",
            "method": "POST",
            "url": "/api/pro/projects/",
            "base_url": "http://124.222.144.182:8888",
            "headers": {
                "Content-Type": "application/json",
                "Authorization": "Bearer {{token}}"
            },
```

```
                "params": {},
                "body": {
                    "name": "TestProject",
                    "user": "{{user_id}}"
                }
            },
            "assertions": {
                "response": [
                    {
                        "type": "http_code",
                        "field": "HTTP状态码",
                        "expected": 201
                    },
                    {
                        "type": "not_null",
                        "field": "id",
                        "expected": True
                    },
                    {
                        "type": "equal",
                        "field": "name",
                        "expected": "TestProject"
                    },
                    {
                        "type": "equal",
                        "field": "user_id",
                        "expected": "{{user_id}}"
                    },
                    {
                        "type": "not_null",
                        "field": "create_time",
                        "expected": True
                    }
                ],
                "database": []
            }
        },
        {
            "name": "删除项目功能测试",
            "description": "测试删除项目功能，验证项目能被正确删除且数据从系统中清除",
            "project": "web测试平台",
            "module": "项目管理",
            "interface": "删除项目",
            "preconditions": [
                {
                    "name": "用户登录",
                    "request": {
                        "interface_id": "/api/users/login",
                        "method": "POST",
                        "url": "/api/users/login",
                        "base_url": "http://124.222.144.182:8888",
                        "headers": {
                            "Content-Type": "application/json"
                        },
                        "params": {},
                        "body": {
                            "username": "python771",
                            "password": "musen12399"
```

```
                        }
                    },
                    "extract": [
                        ["token", "token"]
                    ]
                },
                {
                    "name": "创建项目",
                    "request": {
                        "interface_id": "/api/pro/projects/",
                        "method": "POST",
                        "url": "/api/pro/projects/",
                        "base_url": "http://124.222.144.182:8888",
                        "headers": {
                            "Content-Type": "application/json",
                            "Authorization": "Bearer {{token}}"
                        },
                        "params": {},
                        "body": {
                            "name": "测试项目_待删除",
                            "user": 1
                        }
                    },
                    "extract": [
                        ["project_id", "id"]
                    ]
                }
            ],
            "request": {
                "interface_id": "/api/pro/projects/{id}",
                "name": "删除项目主请求",
                "method": "DELETE",
                "url": "/api/pro/projects/{{project_id}}",
                "base_url": "http://124.222.144.182:8888",
                "headers": {
                    "Authorization": "Bearer {{token}}"
                },
                "params": {},
                "body": {}
            },
            "assertions": {
                "response": [
                    {
                        "type": "http_code",
                        "field": "HTTP状态码",
                        "expected": 200
                    },
                ],
                "database": [

                ]
            }
        }

    ]

    result = tool.invoke({"test_data": test_data})
    print("执行结果:", result)
```

三、Agent决策引擎

1、使用langchain自带的agent

```
# @Author  : 木森
# @weixin: python771
"""
API测试Agent
整合所有工具功能，包括：
1. 向量检索
2. 基础用例生成
3. 结构化用例生成
4. 数据库查询
5. 用例执行
"""

from langgraph.prebuilt import create_react_agent
from agents.models import qwen3_llm
from tools import (
    VectorRetrieverTool,
    BaseCaseGeneratorTool,
    OneStructuredCaseGenerationTool,
    BatchStructuredCaseTool,
    CompleteCaseGenerationTool,
    InterfaceQueryTool,
    BaseTestCaseQueryTool,
    StructuredTestCaseQueryTool,
    CaseExecutionTool
)


class APITestAgent:
    """API测试Agent"""

    def __init__(self):
        """
        初始化Agent
        :param llm_model: LLM模型名称
        """
        # 初始化LLM
        self.llm = qwen3_llm
        # 初始化工具
        self.tools = [
            # 向量检索工具
            VectorRetrieverTool(),
            # 基础用例生成工具
            BaseCaseGeneratorTool(),
            # 单个结构化用例生成工具
            OneStructuredCaseGenerationTool(),
            # 批量结构化用例生成工具
            BatchStructuredCaseTool(),
            # 完整用例生成工具
```

```
            CompleteCaseGenerationTool(),
            # 接口查询工具
            InterfaceQueryTool(),
            # 基础用例查询工具
            BaseTestCaseQueryTool(),
            # 结构化用例查询工具
            StructuredTestCaseQueryTool(),
            # 用例执行工具
            CaseExecutionTool()
        ]
        # 创建Agent
        self.agent = create_react_agent(
            llm=self.llm,
            tools=self.tools, )


if __name__ == '__main__':
    agent = APITestAgent()
    print(agent.agent.invoke({"messages": [{"role": "user", "content": "获取登录接口的测试用例"}]}, ))
```

2、自定义Agent决策引擎

- **Agent 决策引擎六大组成**

构成模块	功能描述
1. 意图识别	解析用户输入,理解用户意图、分析上下文
2. 状态追踪与记忆	保持任务上下文、历史状态
3. 决策推理	判断行动策略、推演步骤
4. 工具调度	选择合适工具并构造调用参数
5. 行为执行	执行行动、调用工具、响应输出
6. 控制策略	控制流程、设定终止条件、多轮决策控制